# BRAHMA + ASTRA

## FINAL DESIGN APPROVAL RECORD

Development kickoff checklist • Locked design tokens • Architecture • Security • Delivery phases

**Approval status:** BRAHMA and ASTRA design directions are approved and locked by the user.

**Change control:** No visible design changes without explicit user approval.

**Record version:** 1.0

**Approval status:** Both product designs approved by the user

**Scope:** Design freeze, implementation constraints, development readiness, and kickoff checklist

# 1. Approval record

## BRAHMA — Approved and locked

The user approved the complete BRAHMA design with no changes. The following remain locked:

• Product name: **BRAHMA**

• Visual direction: premium dark cosmic interface with gold lotus/mandala identity and restrained glassmorphism

• Tagline: **Think Deeper • Create Faster • Build Bigger**

• v1 color tokens and typography as recorded below

• Home Dashboard v1 layout

• Complete screen-review set as approved

• No design changes unless explicitly requested by the user

## ASTRA — Approved and locked

The user approved **ASTRA v1 as presented**. The following remain locked:

• Product name: **ASTRA**

• Descriptor: **CODE · CREATE · DEPLOY**

• Dedicated Indian-mythology-inspired celestial-arrow/astral emblem direction; do not reuse BRAHMA's lotus as ASTRA's logo

• ASTRA-specific palette below

• Separate developer-focused workspace layout

• Proposed six-stage, approximately 5–7 second opening-animation storyboard, including skip and reduced-motion behavior

• No design changes unless explicitly requested by the user

**Important asset note:** Approval locks the presented design direction and specification. Clean production SVG logo assets, motion-ready vector layers, implemented animation timing, and browser screenshot validation still need to be produced and verified during implementation. The concept poster itself is not a vector logo or a running application screenshot.

# 2. Locked BRAHMA design tokens

| Token | Value | Use |
|---|---|---|
| `brand.gold` | `#F6C87A` | Brand mark and emphasis |
| `brand.goldLight` | `#FFE5A7` | Gold highlights |
| `brand.violet` | `#8B5CF6` | Main interaction/selection |
| `brand.cyan` | `#06B6D4` | Technical/status accent |
| `brand.purple` | `#A855F7` | Secondary glow |
| `bg.base` | `#0B0F1A` | Main background |
| `bg.elevated` | `#111827` | Elevated panels |
| `surface.card` | `#1E1B3D` | Cards/glass panels |
| `text.primary` | `#F7F7FB` | Primary text |
| `text.secondary` | `#A7AEC2` | Secondary text |
| `border.default` | `#30384D` | Borders |
| `status.success` | `#22C55E` | Success |
| `status.warning` | `#F59E0B` | Warning |
| `status.error` | `#EF4444` | Error |
| `font.ui` | `Poppins` | UI |
| `font.code` | `JetBrains Mono` | Code/terminal |

# 3. Locked ASTRA design tokens

| Token | Value | Use |
|---|---|---|
| `astra.bg` | `#070A18` | Main IDE background |
| `astra.surface` | `#0D1428` | Sidebar/panels |
| `astra.panel` | `#101B35` | Elevated work panels |
| `astra.violet` | `#8B5CF6` | Primary interactive accent |
| `astra.electricBlue` | `#38BDF8` | Code/technical highlights |
| `astra.cyan` | `#22D3EE` | Status/secondary actions |
| `astra.gold` | `#F6C87A` | Restrained shared-family highlight |
| `astra.text` | `#F7F7FB` | Primary text |
| `astra.muted` | `#A7AEC2` | Secondary text |
| `astra.border` | `#263657` | Panel borders |
| `astra.success` | `#22C55E` | Passing checks |
| `astra.warning` | `#F59E0B` | Warnings |
| `astra.error` | `#EF4444` | Failures |
| `font.ui` | `Poppins` | UI |
| `font.code` | `JetBrains Mono` | Editor/terminal/diffs/logs |

# 4. ASTRA opening storyboard — locked concept

Proposed duration: approximately **5–7 seconds**, with Skip available after the emblem reveal.

**Handoff (0.0–0.6 s):** User selects ASTRA; BRAHMA dims smoothly.

**Celestial field (0.6–1.6 s):** Indigo field with fine violet/blue orbital geometry.

**Energy gathers (1.6–2.6 s):** Light points converge; symbolic celestial arrow forms.

**Emblem reveal (2.6–3.6 s):** ASTRA emblem and `CODE · CREATE · DEPLOY` appear.

**Transition (3.6–4.6 s):** Light trail leads into the workspace.

**Workspace materializes (4.6–6.0 s):** Explorer, editor, assistant, and terminal appear.

**Ready state:** Show readiness only after project context/services actually initialize; otherwise show loading/error and retry.

Motion rules: reduced-motion alternative; no flashing/strobing; sound off by default if ever added; do not trap the user in decorative animation; never falsely indicate readiness.

# 5. Shared product decisions

• BRAHMA is the main intelligence platform; ASTRA is its separate coding/development workspace.

• Frontend: React + JavaScript; **no TypeScript files**.

• Application backend: Node.js + JavaScript + Express.

• AI backend: Python + FastAPI. Brahma Core handles orchestration; Astra Engine handles coding-specific workflows.

• Data: PostgreSQL; Redis for queue/cache/rate limiting; private object storage for uploads/artifacts.

• Start with model-provider adapters so providers can be changed without rewriting application logic. Keep credentials server-side.

• Web first; Electron may be added later, sharing frontend and services.

• Do not use provider brand names as primary sidebar agent identities.

• Do not reuse old FRIDAY code or introduce OpenJarvis unless explicitly requested. Clean-slate implementation.

• Do not fabricate usage metrics, billing, model availability, test results, or deployment success.

# 6. Development kickoff checklist

## A. Repository and project setup

■ Create a clean repository/workspace for BRAHMA + ASTRA.

■ Establish branch protection, pull-request reviews, commit conventions, and version tags.

- Create frontend, application API, AI services, shared contracts, infrastructure, and documentation directories.
- Add `.editorconfig`, `.gitignore`, lint/format rules, pre-commit checks, and CI.
- Pin supported Node.js and Python versions; document local setup.
- Add `.env.example` with placeholders only; never commit real secrets.

## B. Frontend foundation

- React + JavaScript (no `.ts`/`.tsx`).
- Install and confirm Poppins and JetBrains Mono licensing/availability.
- Build centralized CSS custom properties for the locked BRAHMA and ASTRA tokens.
- Create approved SVG/logo assets and favicon/app icons from the locked references.
- Build shared accessible primitives: buttons, inputs, dialogs, cards, tooltips, tabs, toasts, loading/empty/error states.
- Implement BRAHMA shell and approved Home Dashboard first.
- Implement BRAHMA screen routes and then separate ASTRA workspace routes.
- Implement ASTRA animation with skip/reduced-motion and accurate readiness states.
- Add responsive and screenshot comparison tests at agreed viewports.

## C. Application API (Node.js + Express)

- Define API versioning, validation schemas, structured error format, request IDs, and OpenAPI documentation.
- Implement authentication/session lifecycle, logout, account recovery, and optional MFA.
- Enforce authorization and resource ownership on every protected operation.
- Implement projects, conversations, uploads, user preferences, memory controls, notifications, and audit event APIs as needed.
- Add rate limiting, payload/file size limits, secure headers, CORS policy, and request logging with secret redaction.
- Keep model credentials and other secrets on the server; add rotation procedure.

## D. AI services (Python + FastAPI)

- Define request/response contracts between Express and FastAPI.
- Build provider adapter interface, timeout/retry policy, error mapping, cancellation, and usage accounting.
- Implement Brahma Core orchestration and product-owned agent routing.
- Implement Astra Engine coding workflow: inspect, plan, propose patch, review, run tests, report.
- Add prompt-injection defenses for user files, repositories, retrieved pages, and tool output.
- Make tool permissions enforceable in code, not only in model prompts.

## E. Data and storage

- Design PostgreSQL schema and migrations before feature implementation.
- Define tenant/user ownership boundaries and indexes.
- Use Redis for queues, caching, rate limiting, and short-lived coordination only where appropriate.
- Store uploads and generated artifacts in private object storage with authorization checks and retention rules.
- Define backup, restore, retention, export, and deletion procedures.
- Avoid storing sensitive prompt/file content in routine logs.

## F. ASTRA execution safety

- Use disposable isolated execution environments; never execute generated code directly on the application host.
- Apply CPU, memory, PID, disk, time, filesystem, and network limits.
- Separate project workspace mounts from service credentials and host filesystem.
- Require approval for destructive changes, risky dependency installation, commits/pushes, secret access, and deployment.
- Show exact command, target, effects, and status before/after execution.
- Implement cancellation, timeouts, output caps, audit logs, and cleanup.
- Treat repository content and external instructions as untrusted.

### G. Security, privacy, and quality

■ Threat-model authentication, uploads, tools, ASTRA sandbox, model providers, and data boundaries.

■ Set a target security baseline such as OWASP ASVS Level 2; verify rather than claim compliance prematurely.

■ Test authorization/IDOR, CSRF where applicable, XSS, SSRF, injection, upload handling, rate limiting, secret leakage, and prompt injection.

■ Add unit, integration, API contract, end-to-end, and sandbox tests.

■ Test accessibility, contrast, keyboard navigation, reduced motion, and responsive behavior.

■ Add observability: structured logs, metrics, traces, alerts, and redaction.

■ Prepare incident response, dependency update, backup restore, and rollback procedures.

### H. Environments and release

■ Local development via documented setup or Docker Compose.

■ Separate development, staging, and production configuration/secrets.

■ CI runs lint, tests, dependency/security scans, and build checks.

■ Stage migrations and test backup/restore before production.

■ Run acceptance checks against the approved design references.

■ Obtain explicit release approval before production deployment.

## 7. Recommended delivery phases

**Foundation:** repository, CI, tokens/assets, API contracts, database migrations, auth/session.

**BRAHMA shell:** approved dashboard, navigation, account controls, shared components and responsive layout.

**Core BRAHMA workflows:** chat, projects, documents, research, image generation integration, memory controls, agents, settings.

**ASTRA workspace:** project chooser, editor shell, assistant planning, diff review, terminal/test panel, animation.

**ASTRA safe execution:** isolated runner, permissions, logs, cancellation, testing and deployment confirmations.

**Hardening:** security/privacy testing, accessibility, performance, observability, backups, recovery.

**Release:** staging acceptance, design screenshot comparison, release approval, monitored production rollout.

## 8. Definition of done

A feature is complete only when its design matches the approved references; controls perform real actions; loading/empty/error/success states exist; API authorization is tested; tests pass and are reported honestly; logs avoid secrets; accessibility/responsive checks are complete; and any deviation from locked design has explicit user approval.

## 9. Approval and change control

**BRAHMA:** Approved and locked.

**ASTRA:** Approved and locked.

**Design changes:** No changes to logo direction, palette, typography, layouts, or animation storyboard without explicit user request/approval.

**Implementation detail:** Technical implementation may refine internal code structure, but must preserve approved behavior and appearance. If a visible design change is required, pause and request approval.

## 10. Immediate first development tasks

Create the clean repository and baseline folder structure.

Add frontend, Express API, FastAPI AI service, PostgreSQL/Redis local development configuration.

Add central design tokens and production logo asset placeholders marked clearly as pending vectorization.

Build the BRAHMA shell and approved Home Dashboard against the locked reference.

Add CI and automated smoke tests before expanding feature scope.